前置知識與本章目標

- 前置知識為 C++ 函式、迴圈、基本數學歸納法、對數與等比級數。
- 本章結束時，要能把輸入、輸出及錯誤條件寫成契約，並用不變量說明演算法正確性。
- 分析問題先指定輸入大小與成本模型，再區分最壞、期望及攤銷成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

問題規格先於程式

- 問題要求「在有序陣列找鍵」，仍缺少重複鍵、查無結果及空陣列的語義。
- 本章定義 `lower_bound(A,x)` 回傳最小 $i \in [0,n]$，使 $i=n$ 或 $A[i] \ge x$。前提是 A 非遞減。
- 後置條件為 $\forall j<i, A[j]<x$ 以及 $\forall j \ge i, A[j] \ge x$。回傳 n 不是錯誤，而是合法的插入位置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二分搜尋的演算法規格

1. 初始化 `l=0,r=n`，候選插入位置位於閉區間 `[l,r]`。
2. 當 `l<r`，取 `m=l+⌊(r-l)/2⌋`。
3. 若 `A[m]<x`，令 `l=m+1`。否則令 `r=m`。
4. 回傳 `l`。每次只查閱合法索引 `m<n`，不讀取 `A[n]`。

候選插入位置為閉區間[l,r]，尚未分類元素為半開區間[l,r)，兩者端點語義不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不變量連接規格與實作

- 迴圈不變量為 `0≤l≤r≤n`、`∀j<l,A[j]<x`、`∀j≥r,A[j]≥x`。
- 若 `A[m]<x`，有序性使所有 `j≤m` 都可排除。另一分支使所有 `j≥m` 都可歸到右側。
- 終止時 `l=r`，左右已分類區間覆蓋全部元素，恰好得到後置條件。
- 終止量 `r-l` 是非負整數，且每輪嚴格下降。部分正確性加上終止性才是總正確性。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

追蹤與邊界案例

搜尋 A=[2,4,4,9,13]、x=4。

輪次	l	r	m	比較與更新
1	0	5	2	4≥4，r=2
2	0	2	1	4≥4，r=1
3	0	1	0	2<4，l=1

結果為 1。空陣列直接回傳 0；x=14 回傳 5。這些案例驗證契約，而非只驗證「找到某個 4」。

二分搜尋的可執行契約與完整邊界

```
lower_bound(A,x):
    l=0; r=len(A)
    while l<r:
        m=l+(r-l)//2
        if A[m]<x: l=m+1
        else: r=m
    return l
```

前提是 A 非遞減且比較符合全序。空輸入 A=[] 時 (l,r)=(0,0)，不進迴圈即回0。對 A=[2,4,4,9,13]、x=14，各輪 (l,r,m) 為 (0,5,2)→(3,5,4)→(5,5,無)，回5；x=1 則 (0,5,2)→(0,2,1)→(0,1,0)→(0,0,無)，回0。兩者都沒有讀 A[5]。

重複鍵規格是「第一個不小於」，不能把任一相等位置視為同一契約。若比較器不具遞移性或輸入未排序，排除整個半區的論證失效。Θ(log(n+1)) 是比較次數緊界；每鍵最多L符號時，最壞比較成本另乘 O(L)。

資料抽象化與表示不變量

- 抽象資料型態 ADT 定義可觀察的值與操作，不指定儲存位置。例如集合只要求會員判定、插入與刪除。
- 表示不變量限制合法的實作狀態。動態陣列可要求 $0 \le size \le capacity$，前 `size` 個位置持有有效元素。
- 抽象函數把合法的實作狀態映射成抽象值。證明操作正確，需同時保留表示不變量和 ADT 語義。
- 同一集合 ADT 可由排序陣列、雜湊表或平衡樹實作，其成本與可支援的順序操作不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

時間成本取決於計算模型

- 指定 `n` 為陣列長度，若每次整數比較與索引讀取為 $O(1)$，二分搜尋最壞時間是 $O(\log(n+1))$。
- 若鍵是長度最多 `L` 的字串，每次字典序比較最壞為 $O(L)$，不能省略成固定時間。
- 若資料在磁碟上，隨機存取可能主導成本。RAM 步數與頁面 I/O 數不是相同指標。
- 實測時間受快取、配置器與編譯器影響。漸近分析回答成長率，不能直接推算毫秒數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

O、Ω、Θ 的量詞

- $f \in O(g)$ 表示存在 $c>0, n_0$，使所有 $n \ge n_0$ 有 $0 \le f(n) \le cg(n)$。
- $f \in \Omega(g)$ 是對應下界；$f \in \Theta(g)$ 表示兩者同時成立。
- $3n+8 \in \Theta(n)$。寫成 $O(n^2)$ 也成立，但不是緊界。
- 「最壞時間」是一個成本函數，例如 $W(n)=\max_{|x|=n} T(x)$。O 本身不表示最壞，也不表示平均。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

由迴圈得到求和式

- 對每個 $i=1..n$ 執行 i 次常數操作，總成本為 $\Sigma i = n(n+1)/2 = \Theta(n^2)$。
- 令 $i=1$，每輪 $i*=2$，直到 $i \ge n$，輪數為 $\lfloor \log_2 n \rfloor$，不是 $n/2$。
- 巢狀迴圈不可只數層數。若內圈次數依序為 $n, n/2, n/4, \dots$，總和小於 $2n$。
- 輸出本身若有 n^2 個值，任何逐值輸出的演算法都需 $\Omega(n^2)$ 次輸出操作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

遞迴時間與遞迴空間

- 合併排序的遞迴式為 $T(n)=2T(n/2)+\Theta(n)$。每層總工作 $\Theta(n)$，共 $\Theta(\log n)$ 層。
- 呼叫堆疊只沿一條活躍路徑，深度 $\Theta(\log n)$。不能把整棵遞迴樹的節點數當成同時存在的框架數。
- 若合併共用一個 n 元素緩衝區，額外陣列空間 $\Theta(n)$，加堆疊後仍為 $\Theta(n)$。
- 區分輸入空間、輸出空間與輔助空間。聲稱原地演算法時要說明是否排除遞迴堆疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最壞、期望與攤銷

名稱	平均所跨越的對象	例子
最壞	固定大小的所有輸入取最大	二分搜尋 $\Theta(\log n)$
期望	明示機率空間	隨機 pivot 快排的期望 $\Theta(n \log n)$
攤銷	一整段合法操作序列	倍增陣列 append 攤銷 $\Theta(1)$

攤銷分析不需要輸入隨機。期望可以對演算法內部亂數取平均，而不假設輸入均勻。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倍增陣列的總成本證明

- 容量不足時從 c 擴成 $2c$。假設單一元素搬移成本有固定上界，初始容量至少為 1。
- 從空陣列連續 append n 次，寫入新元素共 n 次，當 $n \geq 2$，搬移數至多為 $1+2+4+\ldots+2^{\lfloor \log_2(n-1) \rfloor} < 2n$；$n \leq 1$ 時從容量 1 開始不需擴容。
- 總成本 $\Theta(n)$，因此 append 攤銷 $\Theta(1)$。某一次擴充仍可能花 $\Theta(n)$。
- 若每次只增加 1 格，搬移總數為 $1+2+\ldots+(n-1)=\Theta(n^2)$。成長策略直接改變攤銷成本。

動態陣列的勢能與單次實際成本

```
append(T value):       // by-value snapshot exists before this body
    if size==capacity:
        allocate max(1,2*capacity) slots
        relocate existing elements; commit storage
    construct from independent value at storage[size]
    size=size+1
```

從容量1空陣列開始，按「寫入一個值成本1、搬移一個值成本1」記帳。對 $n \geq 1$ 且 $c \leq 2n$ 的可達狀態，取 $\Phi = 2n - c$；第一次 append 可另付常數以處理初始 $\Phi = -1$。未擴容時實際成本1、$\Delta\Phi = 2$，攤銷3。滿容量 $n = c$ 擴到2c再加入一值，實際c+1，新勢能2，舊勢能c，故攤銷仍3。配置成本若與容量線性同階，只改固定係數。

具體序列的 (n,c,實際成本) 為 (1,1,1)、(2,2,2)、(3,4,3)、(4,4,1)、(5,8,5)。某次成本5不推翻固定攤銷界。若每次刪除都縮到恰好size，交替push/pop可能不斷搬移，前述只成長的勢能分析不能直接沿用；需保留分離的擴縮門檻。

動態陣列的勢能與單次實際成本 （續）

此處的append(T value)明定為「先建構獨立的值參數，再進入函式本體」，不是const T&。因此即使呼叫append(A[0])且A已滿，value也在舊storage失效前完成快照，擴容後從獨立value建構新元素。最小例容量1、內容[7]：先快照7，再搬移原7及釋放舊區，最後得到[7,7]。若改收const T&並先grow，參照原A[0]的參數會懸空，不能再讀。

本節限定可複製建構的T，參數快照成本C_arg另計；上述攤銷3只計原定義的儲存寫入／既有元素搬移。若每次快照也收1單位，改為攤銷至多4；若複製長字串需Θ(L)，就另加該成本，不能稱整次append固定常數。參數建構失敗發生在本體之前，陣列尚未修改。

非均一寫入成本的計數器勢能

```
increment(A):
    require A is not the maximum k-bit value
    i=0
    while A[i]==1: A[i]=0; i+=1
    A[i]=1
// A[0] is the least significant bit.
// Read cost: 1; write cost at i: c_i=(3/2)^i.
// Phi(A)=sum_{i=0}^{k-1} (1+3*(3/2)^i)*A[i].
```

攤銷不是把較慢操作忽略，而是把未來可預見的支出存在目前狀態。此題模型只計每次 bit 讀取 1 與寫入 $c_i = (3/2)^i$；其他固定指令暫不計。從全 0 開始，不執行溢位那一步。若低 r 位都是 1，當次讀 $r+1$ 位，寫第 $0, \dots, r$ 位，實際成本為 $C_r = (r+1) + \sum_{i=0}^{r} (3/2)^i = r - 1 + 3(3/2)^r$。

取明確非負勢能 $\Phi = \sum_{i=0}^{k-1} a_i A[i]$，其中 $a_i = 1 + 3(3/2)^i$，不留待選的常數。當次低 r 位由 1 變 0，第 r 位由 0 變 1，故 $\Delta\Phi = a_r - \sum_{i=0}^{r-1} a_i = 7 - r - 3(3/2)^r$。兩式相加得到每次攤銷成本恰 6，與 k, r 無關。從全 0 有 $\Phi_0 = 0$，T 次操作總實際成本為 $6T - \Phi_T \le 6T$。

非均一寫入成本的計數器勢能 （續）

實例以低位在右的二進位顯示：000→001，C=2、ΔΦ=4；001→010，C=9/2、ΔΦ=3/2；010→011，C=2、ΔΦ=4；011→100，C=31/4、ΔΦ=−7/4。各次C+ΔΦ皆6。四次實際總65/4，最後Φ=31/4，和為24。若只用「1的個數」作勢能，昂貴高位支出沒有足夠權重支付。

由未來支出反推係數及成長臨界

```
// General processed-bit charge: q_i=b+alpha^i, 0<alpha<2, b>=0.
K=sum_{j=0}^infinity q_j/2^j = 2*b+2/(2-alpha)
a_i=2^i*sum_{j=i+1}^infinity q_j/2^j
   =b+alpha^(i+1)/(2-alpha)
Phi(A)=sum_i a_i*A[i]
```

令每個處理位的完整記帳為 $q_i = b + \alpha^i$，其中 b 明確計入讀取與選定的固定控制成本。要求清 r 位再設第 r 位的攤銷成本都等於 K，便有 $a_r = K - \sum_{i=0}^{r} q_i + \sum_{i=0}^{r-1} a_i$。遞迴展開為 $a_r = 2^r (K - \sum_{j=0}^{r} q_j / 2^j)$。選 $K = \sum_{j=0}^{\infty} q_j / 2^j$，剩下的尾和非負，恰得到上方係數；不是先猜一個勢能再省略如何選常數。

α<2使幾何和收斂，係數都是可計算的實數；前頁b=1、α=3/2得到K=6、a_i=1+3(3/2)^i。若每個處理位另需恰3單位控制成本，就改b=4，取K=12及a_i=4+3(3/2)^i，仍是明確常數。任意起始bit配置的總成本界另加Φ初值，不能保證與初始狀態無關。

臨界反例 $\alpha = 2$：從 0 走到全 1 的 $2^k - 1$ 次操作，第 i 位共寫 $2^{k-i} - 1$ 次，因此總寫成本為 $\sum_{i=0}^{k-1} 2^i (2^{k-i} - 1) = k 2^k - (2^k - 1)$，平均 $\Theta(k)$，不存在與 k 無關的總序列常數界。這裡由真實總成本否定保證，不只是說某個候選勢能失敗。固定 k 時仍是有限常數，但分析要求對可變 k 一致。

來源與授課邊界

- 本章的二分搜尋追蹤、契約寫法、求和與倍增推導為原創教學編排。
- C++ `vector` 的末端操作攤銷複雜度與連續儲存要求，核對 C++ 工作草案 vector overview。標準不承諾固定採用倍增倍率。
- 公開課程脈絡參見 MIT 6.006 2020 講義索引，L1–L2。
- 未核對指定參考書本章實際節次與頁碼。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study